A PROJECT REPORT
on

QLens: An Automated Relational Database Design and Normalization System

Submitted to Manipal University Jaipur
Towards the partial fulfillment for the Award of the Degree of

B. Tech Computer Science and Engineering

2026–2027

By

Aditya Gaur — Reg. No. 2502051500

Aditya Choudhary — Reg. No. 2502051502

Semester III, Section K3 · Relational Database Management Systems



MANIPAL UNIVERSITY
JAIPUR

Under the guidance of

Ms. Surabhi Agrawal

Department of Artificial Intelligence and Machine Learning
Manipal University Jaipur

Jaipur, Rajasthan

Abstract

Many small offices keep all their records in one large spreadsheet. A coaching institute's fee register, for example, repeats every teacher's phone number on hundreds of rows and spells some students' names in more than one way. In database terms the sheet is an unnormalized relation, and it suffers from update, insertion and deletion anomalies. The people who use it do not know how to design a database, so the problem stays.

QLens is a web application that designs a relational database *from the data in the sheet*. It loads the sheet into PostgreSQL, finds the functional dependencies and candidate keys by running SQL queries on the data, and helps the user correct spelling variants and conflicting values. From the dependencies it computes a canonical cover and applies the 3NF decomposition algorithm, which gives tables that are in third normal form, preserve the dependencies and have a lossless join. QLens then creates these tables with primary keys, foreign keys and `UNIQUE` constraints, copies the data into them inside a single transaction, and keeps the result only if joining the new tables back together gives exactly the original rows. Afterwards the user can browse and edit the new tables, where the database's constraints reject bad changes and a trigger records every change, and can run SQL as a restricted database user.

QLens stores its own data in a normalized PostgreSQL schema that uses most topics of the course: an ER model converted to relational tables, keys and integrity constraints, views, triggers, cursors, stored functions, indexes, transactions and GRANT-based access control. On the sample fee register (3,000 rows) QLens finds every planted error and produces the expected four tables (students, batches, teachers and fee records), and rebuilds all 3,000 rows exactly. 507 backend and 204 frontend automated tests pass.

Keywords: relational database design, functional dependency, normalization, third normal form, lossless join, dependency preservation, SQL, PostgreSQL.

Table of Contents

Abstract i

Table of Contents ii

List of Figures iii

List of Tables iv

1 Introduction 1

1.1 Scope of the Work 1

1.2 Product Scenarios 2

2 Requirement Analysis 3

2.1 Functional Requirements 3

2.2 Non-functional Requirements 3

2.3 Use Case Scenarios 3

2.4 Other Software Engineering Methodologies 4

3 System Design 7

3.1 Design Goals 7

3.2 System Architecture 7

3.3 Detailed Design Methodologies 8

4 Conclusion 23

References 24

A More Screens 25

List of Figures

Figure 2.1 Use-case diagram 5

Figure 3.1 System architecture 7
Figure 3.2 ER diagram of the QLens database 9
Figure 3.3 Relational schema of QLens 10
Figure 3.4 From sheet to normalized tables 14
Figure 3.5 Fee-register dependency diagram 14
Figure 3.6 Analyze screen 15
Figure 3.7 ER diagram of the generated fee-register database 18
Figure 3.8 Design screen 18
Figure 3.9 Data page: Browse 19
Figure 3.10 Data page: a constraint error 20
Figure 3.11 Step through 21
Figure 3.12 Running a query 21

Figure A.1 Design after Apply 25

List of Tables

Table 1.1 Part of the fee register 1

Table 2.1 Functional requirements 3
Table 2.2 Non-functional requirements 4
Table 2.3 Test summary 6

Table 3.1 Foreign keys 11
Table 3.2 SQL topics used in QLens 12

CHAPTER 1

Introduction

A *database management system* (DBMS) stores data in tables that are designed so that each fact is kept once. Many small offices do not use one; they keep everything in a single spreadsheet. A coaching institute's fee register is a typical example. Each row records one student's payment for one month:

Table 1.1: Four rows of the sample fee register (3,000 rows in all).

Student Name	Student Phone	Batch	Teacher	Teacher Phone	Month	Fee Paid	Mode
Neha Sharma	+91 98364 94403	JEE Morning	Mr. Gupta	+91 98100 99009	Apr	4000	Cash
Diya Patel	+91 90943 46358	JEE Evening	Mr. Gupta	+91 98100 99009	Apr	3000	Cash
Riya Sharma	+91 99909 46477	JEE Evening	Mr. Gupta	+91 98100 11001	Jun	4000	Cash
riya sharma	+91 99909 46477	JEE Evening	Mr. Gupta	+91 98100 11001	Jul	4000	Cash

The sheet is an *unnormalized relation*: one table holds facts about students, batches, teachers and payments together. This causes the classic problems taught in normalization [1]:

- **Update anomaly.** Mr. Gupta's phone number is stored on every row of every one of his students. When it changed, some old rows were missed, so the sheet now has two numbers for him.
- **Inconsistent data.** "Riya Sharma" and "riya sharma" are the same student written in two ways, so counts and totals are wrong.
- **Insertion and deletion anomalies.** A new teacher cannot be recorded until a student pays, and deleting the last row of a batch loses the only record of who teaches it.

The users of such sheets do not know how to design tables, and existing tools do not help: database front-ends such as Airtable expect the user to design the tables, and chat assistants answer one question but do not leave a correct database behind.

QLens solves this. It loads the sheet into PostgreSQL and finds, from the data itself, which columns decide which other columns (*functional dependencies*) and which columns identify a row (*candidate keys*). It then designs tables in third normal form, creates them with keys and foreign keys, copies the data, and checks that no row was lost. The **objectives** were:

O1. Find functional dependencies and keys using SQL inside the database.

O2. Produce tables in 3NF whose decomposition is lossless and dependency-preserving, and check the lossless join on the real rows.

O3. Keep every user's data separate using database users (roles) and `GRANT`.

O4. Explain every result in plain language.

1.1 Scope of the Work

In scope.

- Import of one CSV or Excel sheet per workspace (up to 100,000 rows and 60 columns).
- Finding functional dependencies and candidate keys, and fixing data errors.
- Designing 3NF tables, creating them and loading the data in one transaction.
- A Data page to browse and edit the new tables, run SQL (with a step-by-step view and index timing) and ask questions in plain English; a history of every query.
- Three sample sheets with known errors: a coaching fee register, a grocery (kirana) shop's sales and a hostel mess register.

Out of scope. Other database systems, several sheets at once, and normal forms beyond 3NF (4NF and 5NF). QLens runs on one computer with its own PostgreSQL 18 server.

Status. Each feature is marked Built (finished and tested), In progress (partly done) or Planned (designed, not started). Nothing is described as working unless it is marked Built.

1.2 Product Scenarios

The institute owner. Built The owner imports the fee register. QLens reports that *(Student Name, Month)* identifies every row, that Batch decides Teacher, and that Teacher decides Teacher Phone except on 5 rows. After the owner accepts the suggested fixes, QLens proposes four tables (students, batches, teachers and fee records) and creates them. A teacher's phone is now stored once, and "who has not paid for October?" is a short SQL query.

The developer. Built A developer must move an old CSV export into an application database. QLens shows the keys and dependencies that really hold in the data and gives the `CREATE TABLE` statements, ready to copy.

The student. Built A student studying normalization opens the hostel sample, sees why Room decides Hostel Block, follows the canonical cover and the 3NF design step by step, and then practises joins and aggregate queries on the result.

CHAPTER 2

Requirement Analysis

The requirements come from the problems of Chapter 1 and from the three sample sheets, whose planted errors are used as acceptance tests.

2.1 Functional Requirements

Table 2.1: Functional requirements and their status.

ID	Requirement	Status
FR-1	**Accounts and workspaces.** Sign up and sign in; each workspace has its own schema and database user.	Built
FR-2	**Import.** Read a CSV or Excel sheet; keep the original cells unchanged; decide each column's data type.	Built
FR-3	**Keys and dependencies.** Find candidate keys (1–3 columns) and functional dependencies using SQL.	Built
FR-4	**Clean-up.** Find similar spellings and conflicting values; let the user fix them; log every change.	Built
FR-5	**3NF design.** Canonical cover, 3NF decomposition, primary and foreign keys, the `CREATE TABLE` script.	Built
FR-6	**Design screen.** Show the design as a diagram; rename tables and columns before applying (the `id` key column cannot be renamed).	Built
FR-7	**Apply.** Create and fill the tables in one transaction; keep them only if the lossless-join check passes.	Built
FR-8	**SQL.** Run `SELECT` queries read-only and confirmed `INSERT`/`UPDATE`/`DELETE` as one transaction; save queries; keep a history of runs.	Built
FR-9	**Browse data.** Add, edit and delete rows through forms; show which constraint rejected a change; record every change with a trigger.	Built
FR-10	**Ask AI.** Answer questions in plain English with read-only SQL; edit the draft design only through checked, undoable actions.	Built
FR-11	**Explain a query.** Step through a query in relational-algebra order; compare its time with and without a B-tree index.	Built

2.2 Non-functional Requirements

2.3 Use Case Scenarios

Figure 2.1 shows the use cases. The main flow is:

1. The user imports a sheet (or a sample).
2. QLens shows keys, dependencies and data problems; the user fixes the problems.

Table 2.2: Non-functional requirements and how they are met.

ID	Quality	How it is met
NFR-1	Security	Each workspace has its own database user that can only read its own schema (`GRANT SELECT`); queries run read-only with a 5-second limit and at most 1,000 rows.
NFR-2	Correctness	The design is in 3NF, lossless and dependency-preserving; the lossless join is also checked on the real rows.
NFR-3	Integrity	Primary keys, foreign keys, `UNIQUE`, `NOT NULL` and `CHECK` constraints are enforced by the database.
NFR-4	Atomicity	Import, every fix and apply each run as one transaction; a failure leaves nothing half done.
NFR-5	Performance	Tested on the bundled samples (up to about 3,000 rows): analysis finishes within 5 seconds (checked by a test). Staging a 100,000-row import takes about 6 seconds.
NFR-6	Auditability	The original rows are never changed; every fix is logged with the old and new value.
NFR-7	Usability	Results in plain English, light and dark themes.

3. QLens proposes a 3NF design; the user may rename tables or columns.
4. The user applies the design: the tables are created, filled and checked in one transaction.
5. The user browses and edits the new tables and queries them with SQL on the Data page.

2.4 Other Software Engineering Methodologies

Incremental, test-first development. QLens was built in small steps in order of value: database and accounts, import, analysis, clean-up, design, apply, querying. For each step the tests were written first. One example: a randomized test showed that the decomposition lost rows in 74 of 3,000 random cases when no table contained a candidate key, which is why the algorithm adds a key table (Algorithm 3.1). The database schema is created by numbered SQL migration files, so every computer gets exactly the same schema.

Test results. Tests run against a real PostgreSQL database because most of the logic is in SQL (Table 2.3).



Figure 2.1: Use-case diagram. The three target groups specialise a signed-in user and inherit the common flow (import, analyse, clean, design, apply, query); each group adds the use cases it motivates. Colours follow the pipeline phases of Figure 3.4.

Table 2.3: Automated tests (final run: 507 backend tests passed, 1 skipped; 204 frontend tests passed).

Area	Tests	What is checked
Schema, accounts, workspaces	39	constraints, triggers, views, database users and privileges
Import	39	file reading, data types, sample data errors
Keys, dependencies, clean-up	53	`check_fd`, `is_unique`, fixes and rollback
Normalization algorithms	19	closure, canonical cover, 3NF; 900 random cases checked for 3NF and lossless join
Design and apply	79	foreign keys, renames, generated SQL, apply, rollback
SQL tab and query runner	73	one statement only, read-only user, time limit, row limit, run history
Data page, writes, step-through, explain, assistant	205	constraint errors, audit trigger, privileges, timeouts, relational-algebra steps, index timing, assistant tools
Frontend	204	screens' data handling and formatting

CHAPTER 3

System Design

3.1 Design Goals

G1. **Correct design.** Tables must be in 3NF, lossless and dependency-preserving, and the lossless join is checked again on the real rows before the tables are kept.

G2. **SQL in plain sight.** All SQL is hand-written (no code generator hides it), and the user can read the SQL that QLens runs.

G3. **Use the database.** Counting, grouping and checking are done by SQL inside PostgreSQL.

G4. **Safe queries.** A user's query can read only that user's workspace, because its database user has no other privileges.

G5. **Nothing is lost.** The original sheet is never changed, and every fix is logged.

3.2 System Architecture

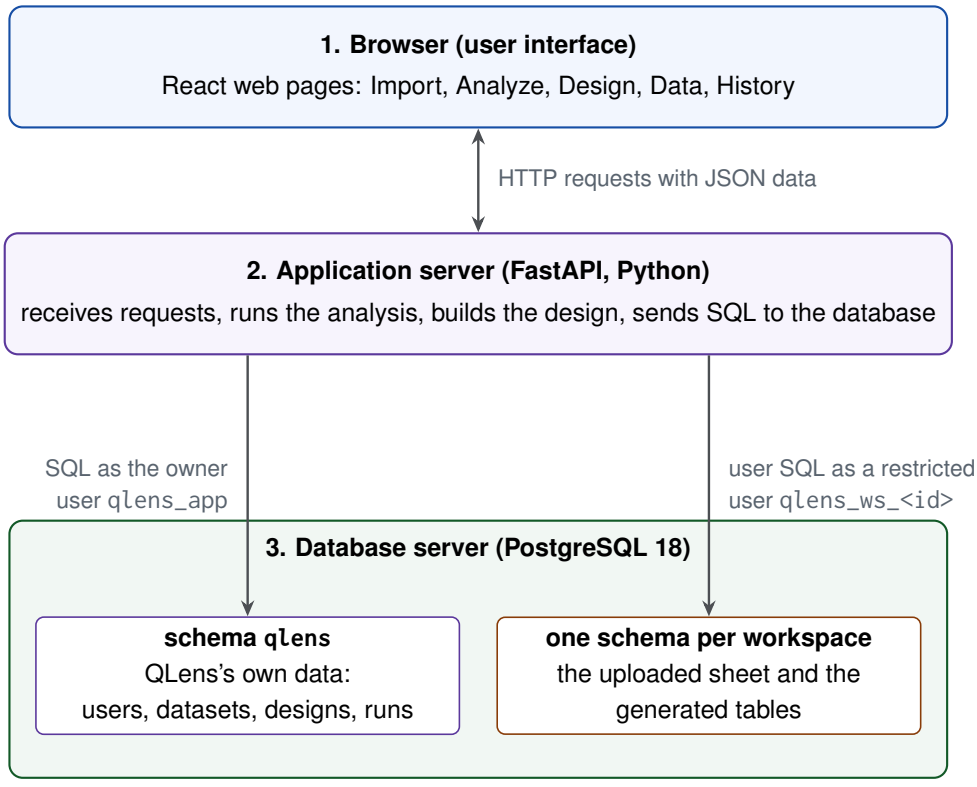


Figure 3.1: Three-tier architecture of QLens. The application talks to PostgreSQL as two different database users: the owner for its own work, and a restricted read-only user (one per workspace) for the queries that people type.

QLens has three tiers (Figure 3.1):

1. **Browser.** The screens are web pages written in React [2]. They send requests to the server and show the results.
2. **Application server.** A FastAPI program in Python [3] receives each request, runs the needed SQL through the psycopg library [4], and returns the answer. Each request runs as one transaction.
3. **Database server.** PostgreSQL 18 [5] holds two kinds of schema (a *schema* is a named group of tables inside one database). The `qlens` schema holds QLens's own data. Each workspace gets its own schemas: `ws_<id>_src` keeps the uploaded sheet (an untouched copy `_raw` and a typed, cleaned copy `_staging`), and `ws_<id>` receives the generated tables.

The application uses two database users. The owner user `qlens_app` does QLens's own work. For every workspace there is also a restricted user `qlens_ws_<id>` that may read that workspace's schemas and change only its generated tables; all SQL typed by people runs as this user (Section 3.3.8).

3.3 Detailed Design Methodologies

This section first describes the database design of QLens itself (ER model, relational schema, constraints, SQL), and then the method QLens uses to design a database for the user, following the fee register as a worked example.

3.3.1 ER Model of QLens

Figure 3.2 is the ER diagram of the data QLens stores. A *User* owns many *Workspaces*; a workspace holds one *Dataset* (the imported sheet), which has many *Columns*. For a dataset QLens finds *Functional Dependencies* and *Candidate Keys*; both are linked to columns by M : N relationships, because one dependency can have several columns on its left-hand side (LHS) and one column can appear in many dependencies. A dataset gets *Schema Proposals* (designs), which contain *Proposed Tables* and *Proposed Columns*; a proposed column may refer to another proposed table (it becomes a foreign key). On the query side, a workspace has *Saved Queries* and *Query Runs*, and a *Row Change* log of every edit made to the generated tables. An *Assistant Conversation* stands for the assistant's three tables (conversations, messages and the tool calls, which may run queries). Some attributes are *derived attributes*: for example a dependency's `is_exact` is derived from `violating_rows = 0`, and PostgreSQL computes it automatically.

3.3.2 ER Diagram to Relational Schema

The ER diagram was converted into tables with the standard rules [1]:

1. Each entity set becomes a table; its key `id` becomes the primary key.
2. A 1 : N relationship becomes a foreign key on the N side (e.g. `workspaces.owner_id` refers to `users.id`).
3. A 1 : 1 relationship becomes a foreign key with a `UNIQUE` constraint (e.g. `datasets.workspace_id`).

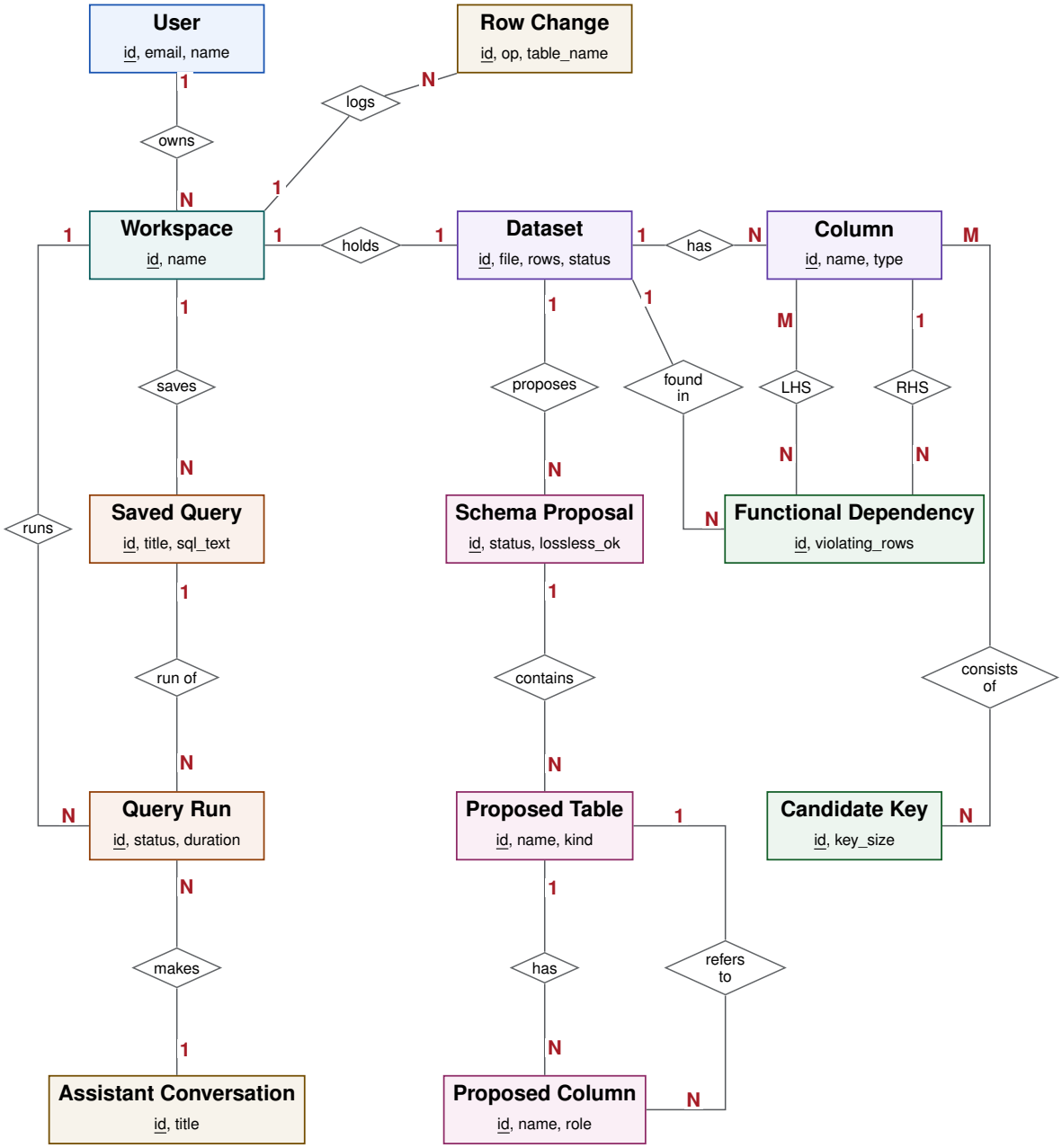


Figure 3.2: ER diagram of the database in which QLens stores its own data (schema `qlens`). Rectangles are entity sets with their primary key underlined, diamonds are relationship sets, and 1, N, M give the cardinality. Only the main attributes are shown; every attribute is listed in the relational schema of Figure 3.3.

4. An M : N relationship becomes its own table whose primary key is the pair of foreign keys (`fd_lhs_columns`, `candidate_key_columns`, `proposed_unique_key_columns`).

Figure 3.3 lists the resulting 26 tables in relational-schema notation.

Accounts and workspaces

users(id, email, password_hash, display_name, created_at)

sessions(token_hash, *user_id*†, created_at, expires_at)

workspaces(id, *owner_id*†, name, created_at, updated_at)

Imported sheet and analysis

datasets(id, *workspace_id*†, original_filename, source, file_size_bytes, row_count, column_count, status, created_at, updated_at)

dataset_columns(id, *dataset_id*†, ordinal, name, label, inferred_type, null_count, distinct_count, sample_values)

candidate_keys(id, *dataset_id*†, key_size)

candidate_key_columns(*key_id*†, *column_id*†)

functional_dependencies(id, *dataset_id*†, *rhs_column_id*†, lhs_size, row_count, lhs_values, repeated_groups, violating_rows, is_exact, violation_ratio, accepted, created_at)

fd_lhs_columns(*fd_id*†, *column_id*†)

duplicate_groups(id, *dataset_id*†, *column_id*†, canonical_value, confidence, max_similarity, status)

duplicate_values(*group_id*†, value, row_count)

dataset_fixes(id, *dataset_id*†, kind, *column_id*†, from_value, to_value, rows_affected, created_at)

Data page

row_changes(id, *workspace_id*†, table_name, op, row_pk, before, after, changed_at)

Proposed designs

schema_proposals(id, *dataset_id*†, *based_on_id*†, status, created_at, applied_at, lossless_ok, staging_rows, reconstructed_rows, mismatch_rows, fd_changes)

proposed_tables(id, *proposal_id*†, ordinal, name, kind, color_index, explanation)

proposed_columns(id, *table_id*†, ordinal, name, *source_column_id*†, data_type, role, not_null, *references_table_id*†)

proposed_unique_keys(id, *table_id*†, is_primary_natural)

proposed_unique_key_columns(*key_id*†, *column_id*†)

Queries

saved_queries(id, *workspace_id*†, title, sql_text, run_count, last_run_at, last_duration_ms, created_at, updated_at)

query_runs(id, *workspace_id*†, *saved_query_id*†, sql_text, source, target, status, duration_ms, row_count, error_message, created_at)

nl_questions(id, *query_run_id*†, question, explanation, model)

explain_snapshots(id, *workspace_id*†, *query_run_id*†, target, plan, planning_ms, execution_ms, created_at)

index_experiments(id, *workspace_id*†, table_name, columns, index_name, ddl, before_ms, after_ms, runs, speedup, status, created_at)

Assistant

assistant_conversations(id, *workspace_id*†, context, title, created_at, updated_at)

assistant_messages(id, *conversation_id*†, seq, role, content, intent, created_at)

assistant_tool_calls(id, *message_id*†, seq, tool, args, ok, result, error, *query_run_id*†, created_at)

Figure 3.3: Relational schema of the `qlens` schema. Primary keys are underlined; foreign keys are in italics and marked †.

3.3.3 Keys and Integrity Constraints

Keys. Every table has a primary key; most use a surrogate key `id` that PostgreSQL numbers automatically (`GENERATED ALWAYS AS IDENTITY`). Natural candidate keys are kept as `UNIQUE` constraints, for example `users(email)`, `workspaces(owner_id, name)` and `saved_queries(workspace_id, title)`. Table 3.1 lists the main foreign keys.

Table 3.1: Main relationships and their foreign keys.

Parent table	Child table (foreign key)	Type	On delete
users	workspaces (owner_id), sessions (user_id)	1 : N	cascade
workspaces	datasets (workspace_id, unique)	1 : 1	cascade
workspaces	saved_queries, query_runs (workspace_id)	1 : N	cascade
datasets	dataset_columns, functional_dependencies, candidate_keys, schema_proposals (dataset_id)	1 : N	cascade
functional_dependencies, dataset_columns	fd_lhs_columns (fd_id, column_id)	M : N	cascade
candidate_keys, dataset_columns	candidate_key_columns (key_id, column_id)	M : N	cascade
schema_proposals	proposed_tables (proposal_id)	1 : N	cascade
proposed_tables	proposed_columns (table_id)	1 : N	cascade
saved_queries	query_runs (saved_query_id, may be NULL)	1 : N	set null

Integrity constraints. The database itself rejects wrong data:

- **NOT NULL** on every column that must always have a value.
- **CHECK** constraints for business rules, for example `CHECK (status <> 'applied' OR lossless_ok IS TRUE)`: a design can be marked *applied* only if its lossless-join check passed; and `CHECK (status IN ('draft', 'applied', 'superseded'))` for allowed values.
- **UNIQUE** constraints that allow only one draft and one applied design per dataset.
- **Referential integrity**: `ON DELETE CASCADE` deletes the children with their parent, so deleting a user leaves no orphan rows; `SET NULL` keeps the run history when a saved query is deleted.

3.3.4 Normalization of the QLens Schema

In most tables every non-key attribute depends on the key, the whole key, and nothing but the key, and every determinant is a key, so these tables are in BCNF (and therefore in 3NF). Two points matter:

1. **No repeating groups.** The columns of a dependency's left-hand side are not stored as a list in one cell; they are rows of the table `fd_lhs_columns`. The same holds for the columns of a candidate key.
2. **Deliberate exceptions.** A few tables keep a value that could be derived, so strictly they have a transitive dependency and are not in 3NF. Each one is a choice, and the database keeps it correct:
   - `dataset_id` in `functional_dependencies`, `duplicate_groups` and `dataset_fixes` is determined by the column (each column belongs to one dataset); it is kept to save a join on the Analyze screen.
   - `is_exact` and `violation_ratio` are derived from `violating_rows` and `row_count`; they are generated columns, which PostgreSQL recomputes on every change.

- A saved query's `run_count` and `last_run_at` summarize its rows in `query_runs`; a trigger updates them, so they cannot disagree.

3.3.5 SQL Used in QLens

Table 3.2 maps the SQL topics of the course to the places where QLens uses them.

Table 3.2: SQL topics of the syllabus and where QLens uses them.

Topic	Use in QLens
DDL	`CREATE TABLE` with keys and constraints for the `qlens` schema and for every generated design; `CREATE SCHEMA`, `DROP SCHEMA` per workspace
DML	`COPY`/`INSERT` to import; `UPDATE` to fix data; `INSERT ... SELECT DISTINCT` to fill the new tables; `INSERT`, `UPDATE`, `DELETE` from the Data page (forms and SQL tab)
DCL	`CREATE ROLE` per workspace; `GRANT SELECT` on its schemas and `GRANT INSERT, UPDATE, DELETE` only on the generated tables; `REVOKE EXECUTE` on the audit function
Joins	rebuilding the original sheet from the new tables; reading each dependency together with its columns
Subqueries	the lossless-join check (Listing 3.2); counting distinct values
Views	`v_workspace_summary` (dashboard), `v_slow_queries` (history page)
Aggregation	`count`, `count(DISTINCT ...)`, `sum`, `max` with `GROUP BY` to find dependencies and keys
Triggers	record every row change on the generated tables (Listing 3.5); keep `updated_at` current; update saved-query statistics
Cursors	the SQL tab fetches results through `DECLARE ... CURSOR` and `FETCH` (Listing 3.6)
Stored procedures	PL/pgSQL functions `check_fd`, `is_unique` and `try_date`
Indexing	B-tree indexes on foreign keys and on `(workspace_id, created_at)`; "Explain & speed up" measures a new index
Transactions	import, every fix, apply and every change on the Data page are single transactions with `ROLLBACK` on failure
Relational algebra	"Step through" shows a query as join, selection, grouping and projection steps

Triggers. QLens uses three triggers: one keeps `updated_at` current, one updates a saved query's run count after each successful run, and one records every change to the generated tables (Listing 3.5 in Section 3.3.8).

Stored function. `check_fd` tests whether a functional dependency $X \to A$ holds in a table. It groups the rows by X and A, and counts how many rows would have to change for every X value to have only one A value. Zero means the dependency holds exactly.

Transactions and ACID. A *transaction* is a group of statements that runs completely or not at all. QLens relies on this:

1. **Atomicity.** Applying a design creates all tables, fills them and checks them in one transaction. If the check fails, `ROLLBACK` removes everything and the old tables stay.
2. **Consistency.** Constraints (keys, `CHECK`, foreign keys) are checked inside the transaction, so a design that breaks them is never committed.

Listing 3.1: Stored function that measures a functional dependency

```
CREATE FUNCTION qlens.check_fd(p_schema text, p_table text, p_lhs text[], p_rhs text)
RETURNS TABLE (row_count bigint, lhs_values bigint, repeated_groups bigint, violating_rows
    bigint)
LANGUAGE plpgsql STABLE AS $$
DECLARE
    t   text := format('%I.%I', p_schema, p_table);
    lhs text;
BEGIN
    SELECT string_agg(format('%I', c), ', ') INTO lhs FROM unnest(p_lhs) AS c;
    RETURN QUERY EXECUTE format(
        'WITH c AS (SELECT %1$s, %2$I, count(*) AS cnt FROM %3$s GROUP BY %1$s, %2$I), '
        '     g AS (SELECT %1$s, sum(cnt) AS n, max(cnt) AS keep FROM c GROUP BY %1$s) '
        'SELECT (SELECT count(*) FROM %3$s), count(*), count(*) FILTER (WHERE n >= 2), '
        '       coalesce(sum(n - keep), 0)::bigint FROM g',
        lhs, p_rhs, t);
END;
$$;
```

3. **Isolation.** Two people applying the same design cannot interfere: the design row is locked with `SELECT . . . FOR UPDATE`.
4. **Durability.** After `COMMIT` the new tables are permanent.

Indexing. PostgreSQL's default index is a B-tree. Besides the indexes it creates for primary keys and `UNIQUE` constraints, QLens adds B-tree indexes on foreign-key columns (for joins and cascading deletes) and an index on `query_runs (workspace_id, created_at DESC)`, which lets the History page read the latest runs of a workspace without scanning the whole table.

3.3.6 Normalization Method (Worked Example)

This is how QLens designs a database for the user, shown on the fee register.

Step 1: find the functional dependencies. A *functional dependency* $X \to A$ means that rows with the same X value always have the same A value. QLens tests every candidate pair with `check_fd` (Listing 3.1) and every small column set with `is_unique` (a candidate key is a minimal set of columns whose values are different in every row). For the fee register it finds the dependencies of Figure 3.5 and the candidate key (`student_name`, `month`). Two dependencies hold on all but a few rows; they point to data errors.

Step 2: clean the data. The 8 rows with name variants ("riya sharma") and the 5 rows with Mr. Gupta's old phone number are shown to the user with a suggested fix (the most common value). Similar spellings are found with PostgreSQL's `similarity()` function. Each fix is an `UPDATE` on the typed copy, run in a transaction and logged. Afterwards both dependencies hold exactly.

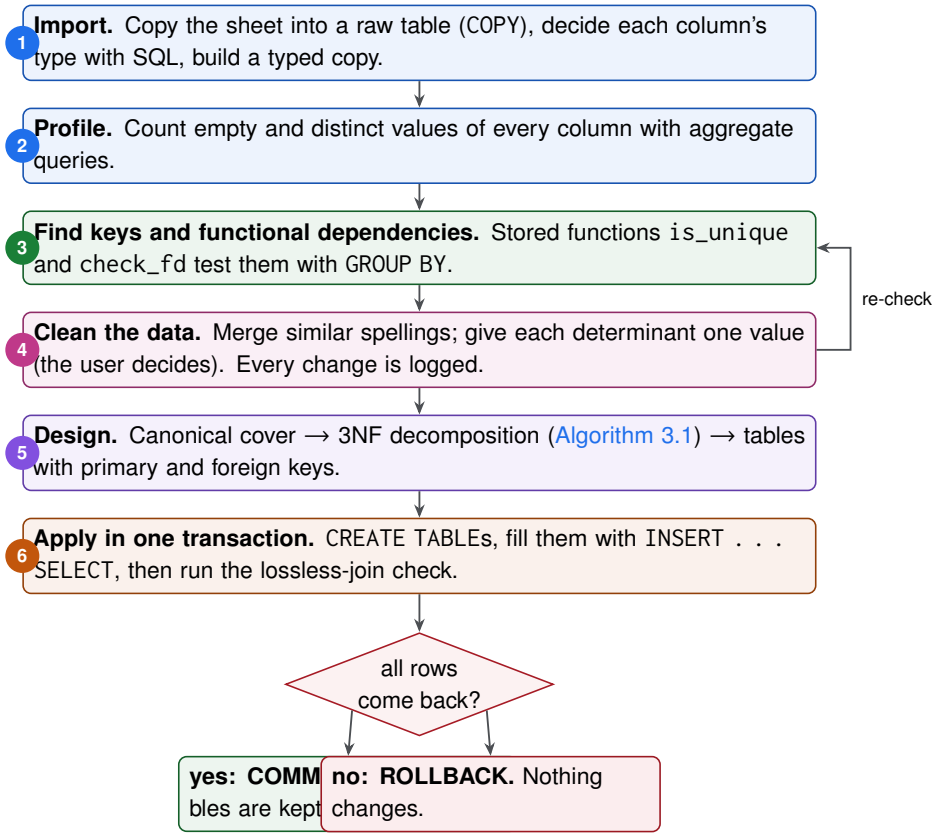


Figure 3.4: The steps from an uploaded sheet to normalized tables. Steps 3 and 4 repeat until the user is happy with the data; step 6 keeps the new tables only if the check passes.

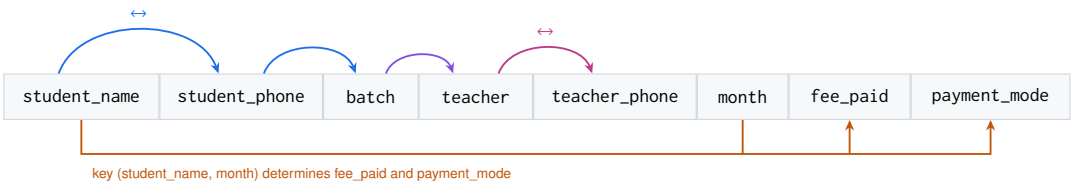


Figure 3.5: Dependency diagram of the fee register: arrows point from determinant to dependent, ↔ marks mutually determining columns, and colours match the tables of Figure 3.7. Before cleaning, {student_phone} → student_name fails on 8 rows and {teacher} → teacher_phone on 5.

Step 3: canonical cover. The *closure* X^+ of a set of columns is every column that X determines through the dependencies. Using closures, QLens removes extra columns from left-hand sides and drops dependencies that follow from the others. This gives the *canonical cover* F_c:

student_name → student_phone, batch student_phone → student_name
batch → teacher teacher → teacher_phone teacher_phone → teacher
student_name, month → fee_paid, payment_mode (from the candidate key)

For example, student_name → teacher is dropped, because it follows from student_name → batch and batch → teacher.

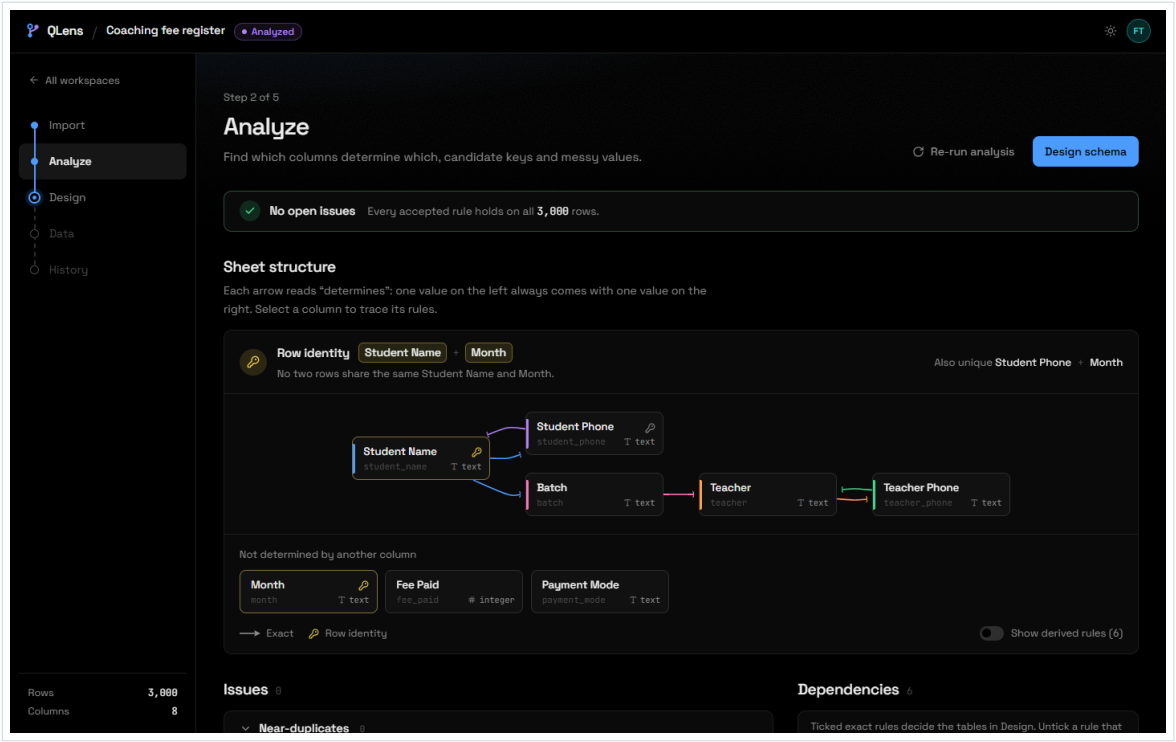


Figure 3.6: Analyze screen after clean-up: the row identity (candidate key Student Name + Month) and the functional dependencies between columns.

Step 4: 3NF decomposition. Algorithm 3.1 makes one table for each dependency of F_c and removes tables contained in others:

- R_1(student_name, student_phone, batch) (student_phone → student_name fits inside R_1)
- R_2(batch, teacher)
- R_3(teacher, teacher_phone) (teacher_phone → teacher fits inside R_3)
- R_4(student_name, month, fee_paid, payment_mode) (contains the candidate key)

QLens then names the tables (students, batches, teachers, fee_register), gives each an `id` primary key, keeps the natural key as `UNIQUE`, and replaces a column that points to another table with a foreign key (batch → `batch_id`, student_name → `student_id`).

Step 5: dependency preservation. Every dependency of F_c lies inside one table, so each can be checked on that table alone; for example the `UNIQUE(teacher)` constraint of `teachers` enforces teacher → teacher_phone. The decomposition is therefore dependency-preserving.

Step 6: lossless join. A decomposition is *lossless* if joining the tables gives back exactly the original relation r. In relational algebra the condition is

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) \bowtie \Pi_{R_3}(r) \bowtie \Pi_{R_4}(r).$$

Algorithm 3.1. 3NF decomposition (synthesis) using a canonical cover, as used by QLens

Input: relation schema R (the sheet's columns); set F of functional dependencies
Output: relation schemas $R_1, \ldots, R_n$ in 3NF; the decomposition is lossless and dependency-preserving

$F_c \leftarrow$ a canonical cover of F
$i \leftarrow 0$
foreach *dependency* $\alpha \rightarrow \beta$ *in* F_c **do**
  $i \leftarrow i + 1$; $R_i \leftarrow \alpha\beta$ // one schema per dependency
if *no* R_j *contains a candidate key of* R **then**
  $i \leftarrow i + 1$; $R_i \leftarrow$ a candidate key of R // guarantees a lossless join
repeat
  if *some* $R_j \subseteq R_k$ $(j \neq k)$ **then** delete R_j // remove redundant schemas
until *no more changes*
return $R_1, \ldots, R_i$

Listing 3.2: Lossless-join check for the fee register (`cols` stands for the eight sheet columns)

```
SELECT (SELECT count(*) FROM (SELECT cols FROM ws_20.original_sheet
                              EXCEPT ALL SELECT cols FROM ws_20_src._staging) d) AS
    extra_rows,
       (SELECT count(*) FROM (SELECT cols FROM ws_20_src._staging
                              EXCEPT ALL SELECT cols FROM ws_20.original_sheet) d) AS
    missing_rows;
```

In theory this holds because R_4 contains a candidate key and each join is on a key of one side. QLens also checks it on the real rows: a view `original_sheet` joins the new tables back, and two set differences must both be empty. `EXCEPT ALL` is SQL's set difference that also counts duplicate rows.

If either count is not zero, the transaction is rolled back. On the fee register both are zero and all 3,000 rows come back.

3.3.7 The Generated Database

Figure 3.7 is the ER diagram of the database QLens builds for the fee register, and Listing 3.3 is part of the DDL it generates.

`UNIQUE NULLS NOT DISTINCT` treats an empty value as one value, so a missing teacher is stored once. `ON DELETE RESTRICT` stops a student from being deleted while fee records still refer to them.

Example queries. Listing 3.4 shows queries a user can run on the new tables in the SQL tab of the Data page. On the sample they return 19 students who did not pay for October, 6 batches with more than 25 unpaid months (the first is JEE Evening of Mr. Gupta, with Rs. 12,75,800 collected) and 124 students who never missed a payment.

Listing 3.3: Part of the DDL generated for the fee register

```
CREATE TABLE ws_20.teachers (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    teacher text NOT NULL,
    teacher_phone text NOT NULL,
    CONSTRAINT teachers_teacher_key UNIQUE NULLS NOT DISTINCT (teacher),
    CONSTRAINT teachers_teacher_phone_key UNIQUE NULLS NOT DISTINCT (teacher_phone)
);
CREATE TABLE ws_20.fee_register (
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    student_id bigint NOT NULL,
    month text NOT NULL,
    fee_paid bigint NOT NULL,
    payment_mode text,
    CONSTRAINT fee_register_student_id_month_key UNIQUE NULLS NOT DISTINCT (student_id,
     month),
    CONSTRAINT fee_register_student_id_fkey FOREIGN KEY (student_id)
        REFERENCES ws_20.students (id) ON DELETE RESTRICT
);
INSERT INTO ws_20.teachers (teacher, teacher_phone)
SELECT DISTINCT s.teacher, s.teacher_phone FROM ws_20_src._staging s ORDER BY 1, 2;
```

Listing 3.4: Example queries on the generated database (join, aggregation, subquery)

```
-- 1. Join: who has not paid for October?
SELECT s.student_name, s.student_phone, b.batch
FROM fee_register f JOIN students s ON s.id = f.student_id
                    JOIN batches  b ON b.id = s.batch_id
WHERE f.month = 'Oct' AND f.fee_paid = 0;
-- 2. Aggregation with GROUP BY and HAVING: collection per batch and teacher
SELECT b.batch, t.teacher, sum(f.fee_paid) AS collected,
       count(*) FILTER (WHERE f.fee_paid = 0) AS unpaid
FROM fee_register f JOIN students s ON s.id = f.student_id
     JOIN batches b ON b.id = s.batch_id JOIN teachers t ON t.id = b.teacher_id
GROUP BY b.batch, t.teacher
HAVING count(*) FILTER (WHERE f.fee_paid = 0) > 25 ORDER BY collected DESC;
-- 3. Subquery: students who never missed a payment
SELECT s.student_name, b.batch
FROM students s JOIN batches b ON b.id = s.batch_id
WHERE NOT EXISTS (SELECT 1 FROM fee_register f
                  WHERE f.student_id = s.id AND f.fee_paid = 0);
```

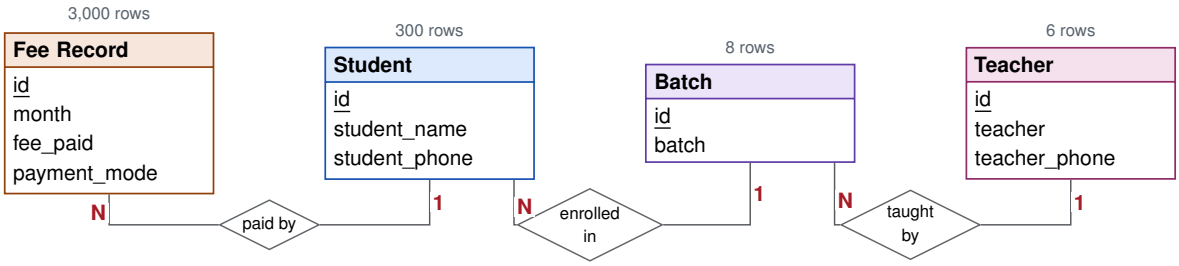


Figure 3.7: ER diagram of the database QLens generates for the fee register. Each fee record is paid by one student, each student is enrolled in one batch and each batch is taught by one teacher (all N : 1). Converted to tables, each N side gets a foreign key (`student_id`, `batch_id`, `teacher_id`).

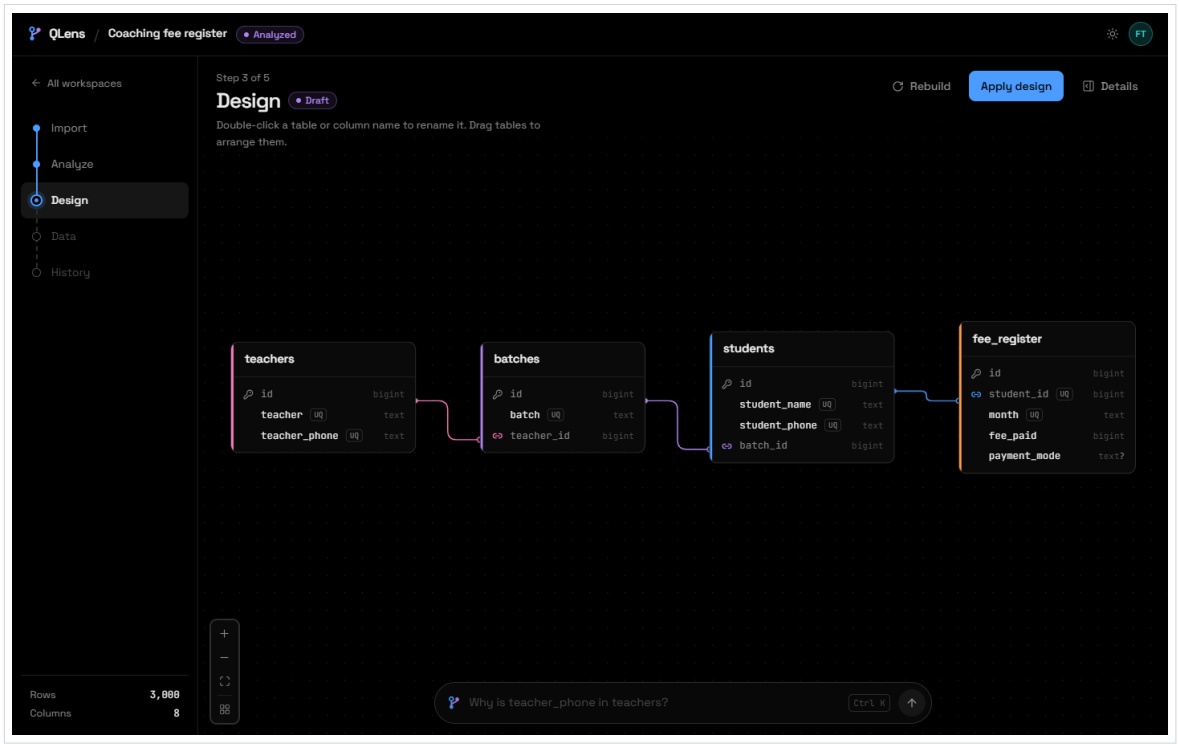


Figure 3.8: Design screen: the proposed tables with their primary keys and foreign keys (the lines), before they are applied.

3.3.8 The Data Page and Security

After a design is applied, the **Data** page lets the user work with the new tables. It has three tabs.

Browse. The user adds, edits and deletes rows through forms, filters rows, follows foreign keys to related records and exports a table as CSV. QLens does not check the rules itself: the database's constraints (`PRIMARY KEY`, `UNIQUE`, `FOREIGN KEY`, `NOT NULL`, `CHECK`) reject a bad change, and the page names the rule that was broken (Figure 3.10). Every change is recorded by a row-level trigger (Listing 3.5): after each `INSERT`, `UPDATE` or `DELETE` on a generated table it stores the old and new row in `qlens.row_changes`, so the page can show a history of edits.

Listing 3.5: Row-level trigger function that records every change to a generated table

```
CREATE FUNCTION qlens.log_row_change() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE
    ws_id   bigint := substring(TG_TABLE_SCHEMA FROM '^ws_([0-9]+)$')::bigint;
    old_row jsonb;
    new_row jsonb;
BEGIN
    IF ws_id IS NULL THEN RETURN NULL; END IF;
    IF TG_OP IN ('UPDATE', 'DELETE') THEN old_row := to_jsonb(OLD); END IF;
    IF TG_OP IN ('INSERT', 'UPDATE') THEN new_row := to_jsonb(NEW); END IF;
    INSERT INTO qlens.row_changes (workspace_id, table_name, op, row_pk, before, after)
    VALUES (ws_id, TG_TABLE_NAME, TG_OP,
            (coalesce(new_row, old_row) ->> 'id')::bigint, old_row, new_row);
    RETURN NULL;
END $$;
-- attached to every generated table when a design is applied:
CREATE TRIGGER log_row_change AFTER INSERT OR UPDATE OR DELETE ON ws_20.students
    FOR EACH ROW EXECUTE FUNCTION qlens.log_row_change();
```

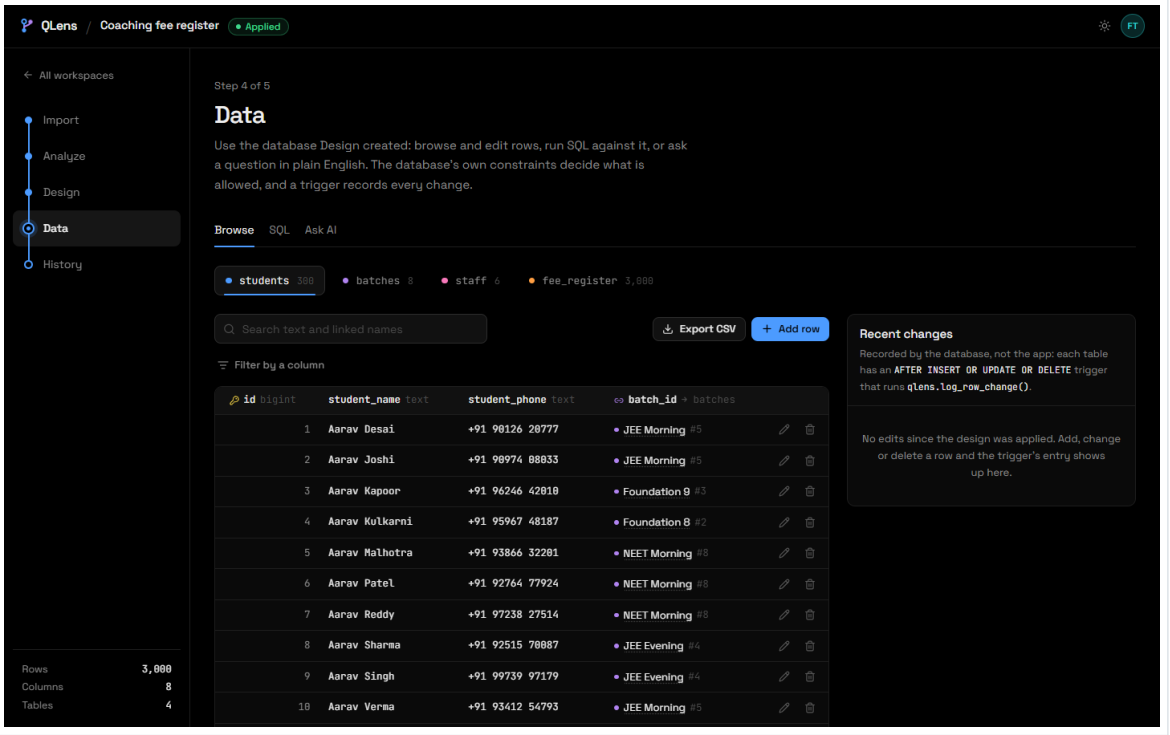


Figure 3.9: Data page, Browse tab: rows of the new tables with their foreign keys, and the history of changes recorded by the trigger.

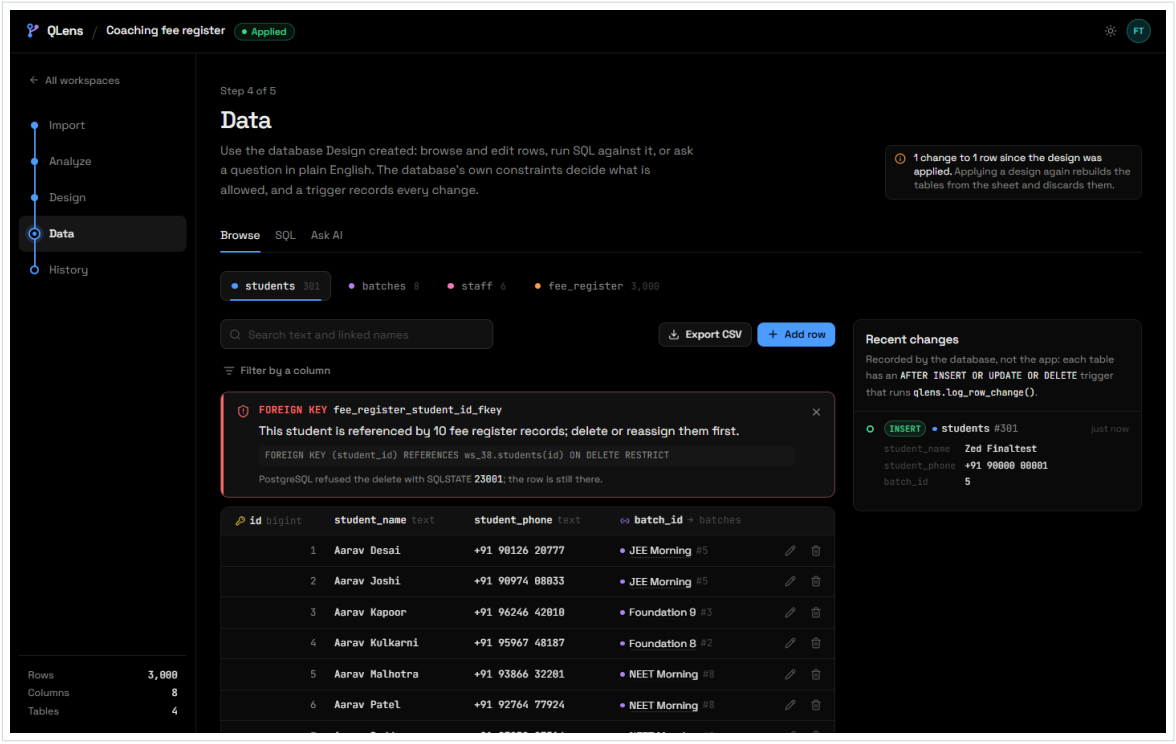


Figure 3.10: Deleting a student who still has fee records: the `FOREIGN KEY` constraint (`ON DELETE RESTRICT`) refuses the change and the page explains why.

SQL. The user can type SQL. A `SELECT` runs read-only; an `INSERT`, `UPDATE` or `DELETE` runs only after the user confirms it, as one transaction. Two extra views explain a query:

- **Step through** shows the query evaluated in relational-algebra order: join, selection (`WHERE`), grouping, `HAVING`, projection and `ORDER BY`, with the number of rows after each step. For "batches with more than 5 unpaid months" the counts are 3,000 → 255 → 8 → 3 (Figure 3.11).
- **Explain & speed up** shows how PostgreSQL executes the query (for example a sequential scan of every row versus an index scan), creates a B-tree index on a copy of the tables and measures how much faster the query becomes.

Ask AI. The user can ask a question in plain English. An optional assistant (Gemini) answers by calling read-only tools that run SQL, and shows the SQL it used. On the Design screen the same assistant can rename tables and accept or reject dependencies; each such action is checked with SQL first and can be undone.

Security with DCL. Users can type any SQL, so the SQL tab must be safe. QLens uses the database's own security:

1. When a workspace is created, QLens runs `CREATE ROLE qlens_ws_<id> LOGIN . . .` and grants it `USAGE` on the workspace's schemas, `SELECT` on their tables, and `INSERT`, `UPDATE`, `DELETE` only on the generated tables. It gets nothing on `qlens` or on other workspaces, so `SELECT * FROM qlens.users` fails with "permission denied".

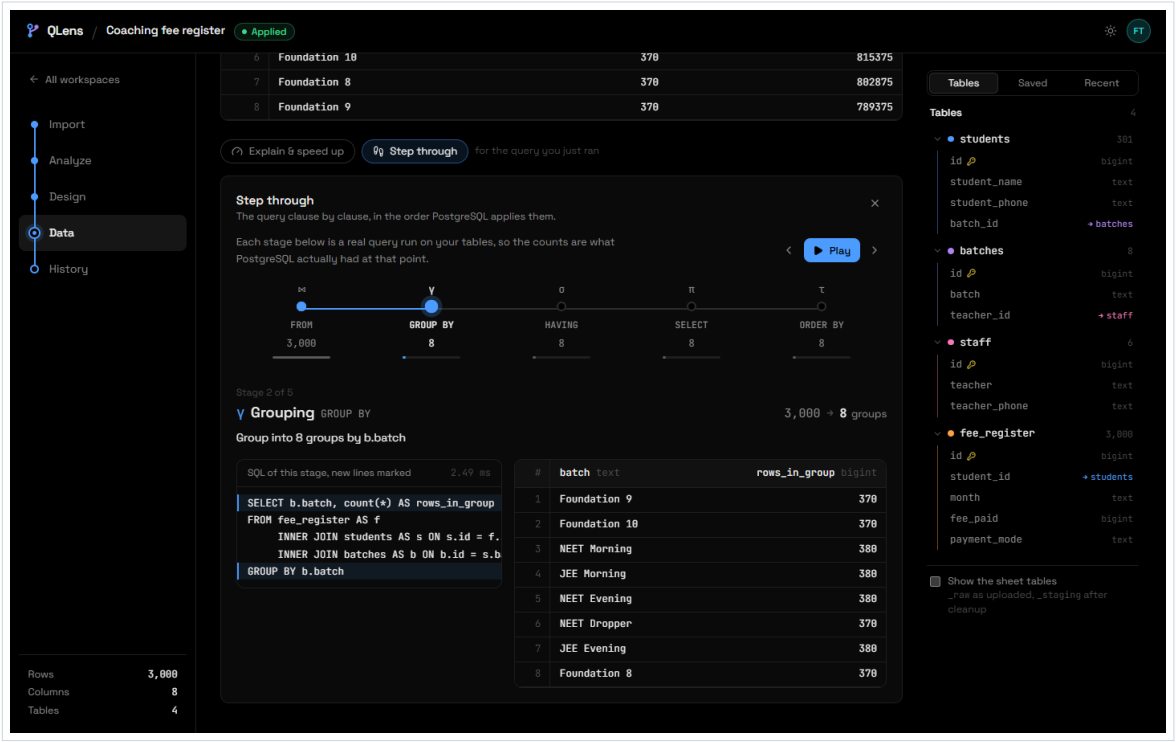


Figure 3.11: Step through: the query evaluated in relational-algebra order; here the grouping step turns 3,000 rows into 8 groups.

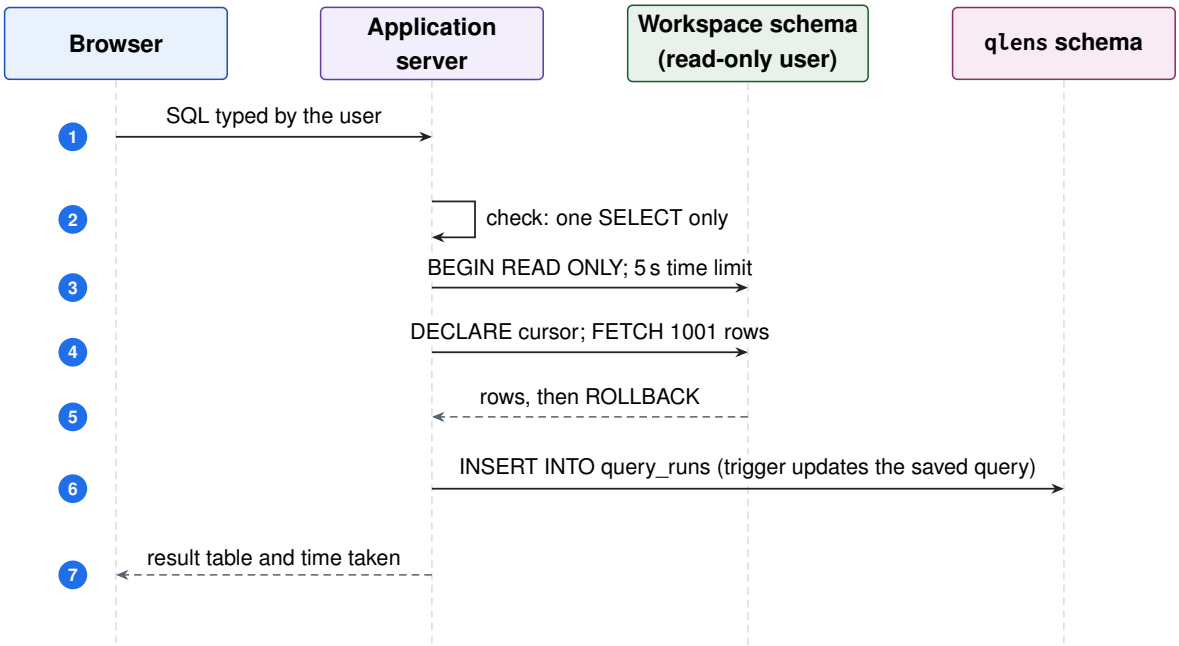


Figure 3.12: Running a `SELECT` typed in the SQL tab. The query is executed through a cursor by a restricted, read-only database user, and every run is recorded in the `qlens` schema.

Listing 3.6: Statements the SQL tab runs for a SELECT, as the workspace's restricted user

```
BEGIN READ ONLY;
SELECT set_config('statement_timeout', '5000', true);
DECLARE qlens_run NO SCROLL CURSOR WITHOUT HOLD FOR <the user's SELECT>;
FETCH FORWARD 1001 FROM qlens_run;
ROLLBACK;
```

2. The role's default is a read-only transaction with a 5-second statement time limit; a confirmed change opens a read-write transaction for that one statement.
3. Before running, QLens checks with the PostgreSQL parser (pglast) that the text is exactly one statement of an allowed kind.
4. A `SELECT` runs through a *cursor*, which returns rows a few at a time (Listing 3.6). QLens fetches 1,001 rows: if the 1,001st exists, the result is marked as cut at 1,000 rows.
5. Every run, successful or not, is stored in `query_runs` and shown on the History page.

3.3.9 User Interface

The app has five steps: Import, Analyze, Design, Data and History. Each generated table keeps one colour on every screen. Figures 3.6 and 3.8 to 3.11 show the main screens with the fee register; more are in the appendix.

CHAPTER 4

Conclusion

QLens shows that a database can be designed from the data itself. Starting from one spreadsheet, it finds functional dependencies and candidate keys with SQL, helps the user correct inconsistent values, computes a canonical cover, and applies the 3NF decomposition algorithm. The resulting tables are in third normal form, preserve the dependencies, and are created and filled in a single transaction that is kept only if the lossless-join check passes on the real rows. On the sample fee register all 13 faulty rows are found and the expected four tables rebuild all 3,000 rows exactly.

QLens's own data is also a normalized relational design, built from an ER model, with primary and foreign keys, `CHECK` and `UNIQUE` constraints, views, triggers, stored functions, a cursor, B-tree indexes, transactions and `GRANT`-based security. 507 backend and 204 frontend tests pass.

What we learnt.

- Real data is never perfect: a dependency may fail on a few rows because of typing errors, so data must be cleaned before it is normalized.
- Letting the database do the work (`GROUP BY`, constraints, transactions) keeps the application simple and the result correct.
- Constraints catch mistakes early; for example, a design cannot be marked applied unless its lossless-join check passed.
- Lossless join, a theorem in the textbook, can be checked with one SQL query.

Limitations.

- Dependencies are searched with at most two columns on the left-hand side and keys with at most three columns; one sheet is normalized at a time and only up to 3NF.
- Analysis was tested on the samples (up to about 3,000 rows). A sheet near the import limit (100,000 rows, 60 columns) can be imported, but its analysis has not been measured and may be slow.
- Query results are cut at 1,000 rows, but not by size, so very wide rows can still make a large result.
- The optional assistant sends the question, the table structure and up to 50 result rows to Google Gemini; the free tier allows only about 20 requests a day.
- The results come from sample data, not a study with real users.

Future work. Support several sheets at once, detect multivalued dependencies for 4NF, and suggest indexes automatically from the query history.

References

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.

[2] Meta Open Source, "React documentation," https://react.dev/, 2025.

[3] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com/, 2025.

[4] D. Varrazzo, "Psycopg 3: PostgreSQL database adapter for Python," https://www.psycopg.org/psycopg3/docs/, 2025.

[5] The PostgreSQL Global Development Group, "PostgreSQL 18 documentation," https://www.postgresql.org/docs/18/, 2025.

CHAPTER A

More Screens

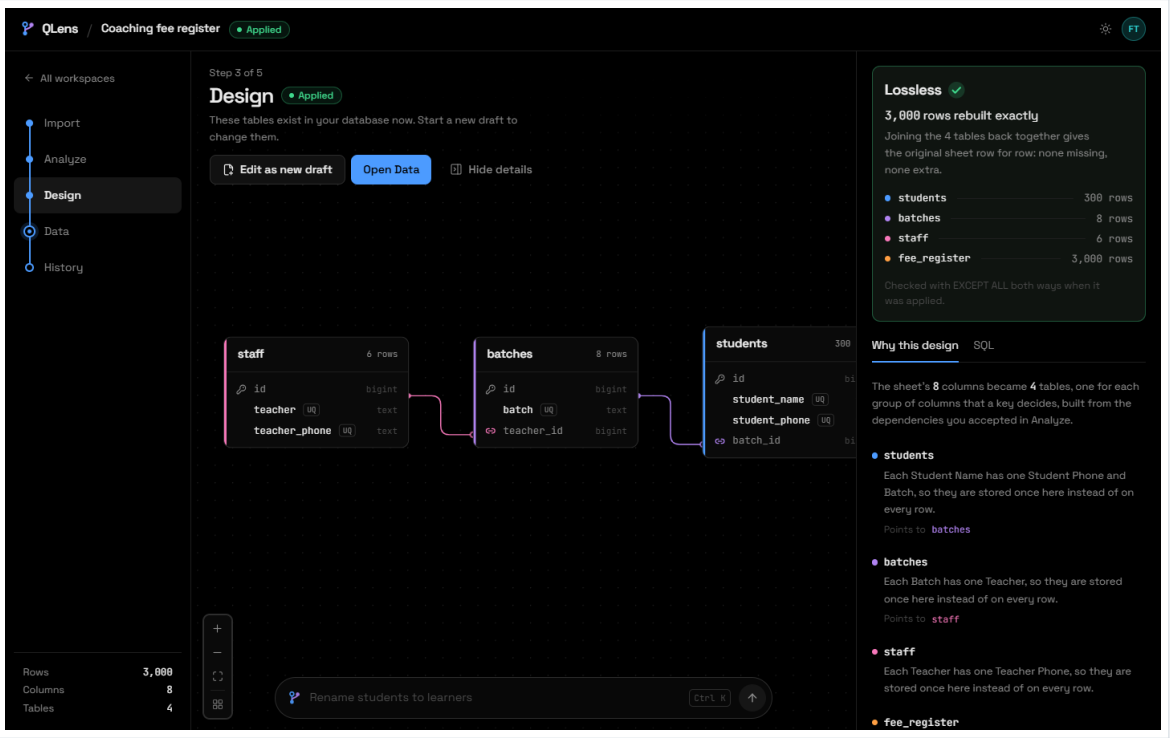


Figure A.1: Design screen after Apply: the lossless-join check rebuilt all 3,000 rows exactly. Here the user had renamed `teachers` to `staff`.